

Shipping it: CLI, MCP, cloud, and design philosophy

Week 9, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- The four deployment surfaces: library, CLI, MCP server, hosted cloud
- browser-use vs Stagehand vs raw Playwright: a decision guide
- The bitter-lesson argument: minimal loop, maximal action space
- When each tool is the right choice

Part 1: four ways to deploy browser-use

Deployment surface 1: Python library

- `pip install browser-use`, the form used throughout this course
- You control the environment, LLM, tools, and output model
- Full visibility into every step of the agent loop
- Right choice: custom integrations, data pipelines, maximum control

Deployment surface 2: the CLI

- Run a task from the command line without writing Python
- Pass the task as an argument or interactively
- Useful for shell scripts, quick one-off automation, non-Python workflows
- Confirm exact syntax at docs.browser-use.com (the interface evolves quickly)

Deployment surface 3: MCP server

- MCP (Model Context Protocol) exposes tools to LLM orchestrators
- browser-use can run as an MCP server, making browser control available to any MCP-compatible agent
- A higher-level agent issues browser commands the same way it would call any other tool
- The consuming agent needs no knowledge of Playwright or CDP
- Confirm exact tool names against the docs, the MCP interface was still evolving as of mid-2026

Deployment surface 4: hosted cloud

- browser-use manages the browser infrastructure for you
- Send tasks via API, receive results, no Playwright setup, no server management
- Right choice: non-Python teams, high-volume workloads, non-technical users
- Trade-off: you are trusting browser-use with any credentials the task requires

Part 2: browser-use vs Stagehand vs Playwright

Raw Playwright: deterministic, zero LLM cost

- You write every click, selector, and navigation
- Reliability on stable sites: ~98% (NxCode, 2026)
- LLM cost: \$0.00
- Failure mode: selectors break when the site changes (15–25% of live sites over 30 days)
- Right choice: high-volume, cost-sensitive pipelines against stable pages

Stagehand: hybrid, TypeScript-first

- Adds AI primitives (`act()` , `extract()` , `observe()`) on top of Playwright
- Developer controls the flow; AI handles targeted natural-language decisions
- Built directly on CDP; TypeScript-only
- Cost: roughly \$0.002–\$0.02 per action (AI invoked only for specific steps)
- Stagehand v3: 44% improvement on complex DOM scenarios (Scrapfly, 2026)
- Right choice: TypeScript ecosystem, frequently changing UIs, surgical AI use

browser-use: fully autonomous, Python-first

- Full agent loop: the LLM decides every step
- Describe the goal; the agent figures out the steps
- Reliability: ~72–78% task completion on real-world tasks (NxCode, 2026)
- Cost: roughly \$0.02–\$0.30 per task
- Supports local LLMs via Ollama for privacy-sensitive workloads (Scrapfly, 2026)
- Right choice: open-ended tasks, rapid prototyping, Python ML/data ecosystem

The decision heuristic

Situation	Tool
Stable site, high volume, cost matters	Playwright
UI changes often, TypeScript team, control the flow	Stagehand
Open-ended task, novel site, Python ecosystem	browser-use
Non-technical user, no infrastructure to manage	browser-use cloud

Part 3: the bitter lesson

Why browser-use is built the way it is

- Most agent frameworks add abstraction layers: pipelines, tool taxonomies, planning steps
- These encode assumptions about how agents should reason
- As models improve, those assumptions become constraints, not help
- The browser-use founders applied Rich Sutton's "bitter lesson" from ML to agent design

The minimal-loop argument

- The actual agent loop is a for-loop: perceive, decide, act, repeat
- Complexity lives in two places only: the action space and the output structure
- `AgentOutput` (Pydantic) forces the model to think in terms of evaluation, memory, goal, and action
- That structure is what makes the loop reliable, not a framework

The maximal-action-space argument

- Most frameworks start with a minimal tool set and expand
- browser-use inverts this: start with full CDP access, restrict for safety
- “As long as in principle everything is possible, LLMs are extremely good at fixing themselves on the fly.” (browser-use team, 2026)
- A model with full browser access can recover from its own mistakes
- A model limited to three pre-approved actions cannot

An agent is just a for-loop of messages... You don't need an agent framework. The less you build, the more it works.

Take-aways

1. browser-use has four deployment surfaces: Python library, CLI, MCP server, and hosted cloud. The right one depends on who runs it and how much infrastructure they want to own.
2. The three-tool decision: Playwright for stability and cost at scale; Stagehand for hybrid TypeScript workflows; browser-use for open-ended Python agent tasks.
3. The bitter-lesson argument: minimize the loop, maximize the action space. Frameworks that add abstraction layers freeze assumptions the model will outgrow.
4. All three tools are real options. The choice is not a debate about “best”, it is a function of task type, language ecosystem, cost envelope, and reliability requirement.

What's next

- **Section (this week):** Run one task three ways and fill in a live scorecard with your own numbers
- **Week 10 (capstone):** Design, build, test, and defend a complete browser-use agent for a real multi-step task
- **Reading:** Week 9 reading covers everything from this lecture plus prompt injection and evaluation methodology in full depth